

SLOT 16

Chapter 12: Versioning, Paging, Sorting and Testing

Part B – Testing • MockMvc + Lab 03 (continued)

CLO3 – Describe RESTful APIs using Spring Boot built-in features and frameworks

JDK 21 • Spring Boot 3 • Maven • JUnit 5 • Mockito • AssertJ • MockMvc

Phạm vi: Chapter 12, slide 20–42 | Không lặp sâu versioning/paging/sorting

Mental model xuyên suốt: Behavior cần bảo đảm → chọn loại test phù hợp → cô lập dependency khi cần → Arrange → Act → Assert → chạy tự động → đọc failure → sửa code, không sửa test để “cho xanh”.

Quality rule: Postman chứng minh “một lần tôi thấy API chạy”. Automated tests chứng minh “mỗi lần code thay đổi, behavior quan trọng vẫn được kiểm tra lặp lại”.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu tiếp tục template Slot 12–15 theo mạch Prepare → Understand → Isolate → Test → Break → Diagnose → Verify → Transfer. Employee Management API của Lab 03 là ngữ liệu xuyên suốt; FUNewsManagementSystem được dùng ở phần transfer.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Chạy lại Lab 03, xác nhận endpoints hoạt động.	Baseline run + Git checkpoint
Understand	Phân biệt unit / slice / integration / E2E.	Testing map
Isolate	Xác định SUT và dependency cần mock.	Dependency sketch
Test	Viết controller/service/repository tests.	Test source
Break	Cố ý làm hỏng status/body/logic.	Red test evidence
Diagnose	Đọc assertion, stack trace, mock interaction.	Debug log
Verify	Chạy toàn suite Maven/IDE.	Green test report
Transfer	Thiết kế strategy cho group project/FUNews.	Project Assistant 02 note

AI rule: AI may suggest tests; student must explain SUT, test boundary, setup, expected behavior và bằng chứng test thật sự fail khi production code bị phá.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1 Ôn lại Slot 15 và Lab 03

- Versioning strategies; paging với Pageable; sorting với Sort; Page vs Slice.
- Lab 03 Employee Management: Controller → Service → Repository; CRUD; paging/sorting; Swagger UI.
- HTTP status codes và JSON response contract.
- Maven lifecycle cơ bản: compile, test, package.

0.2 Công cụ cần chuẩn bị

Công cụ	Vai trò	Check
JDK 21	Compile/run test	java -version
Maven	Dependency + test lifecycle	mvn -v
IntelliJ IDEA	Run/debug tests	Test gutter icon hoạt động
spring-boot-starter-test	JUnit/AssertJ/Mockito/Spring Test	Maven dependency resolved
Postman/Swagger	Manual baseline	Endpoints đang chạy
Lab 03 repo	SUT	git status sạch trước khi test

0.3 Pre-class 3-2-1

- 3 behavior quan trọng của Employee API cần test tự động.
- 2 điểm khác giữa manual API test và automated test.

- 1 câu hỏi về MockMvc hoặc mocking.

0.4 Baseline checkpoint

```
mvn -q -DskipTests compile
mvn test
# Nếu chưa có test, BUILD SUCCESS vẫn có thể xảy ra; hãy quan sát "Tests run".
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Từ Slot 15 sang Slot 16

Slot 15 đã có	Giá trị	Câu hỏi Slot 16 trả lời
Paging	Giảm response lớn	Làm sao tự động kiểm tra page content/metadata?
Sorting	Client kiểm soát thứ tự	Làm sao test asc/desc và invalid sort?
Page vs Slice	Trade-off metadata/performance	Làm sao test contract khác nhau?
Lab 03 API	Endpoint chạy được	Làm sao bảo vệ behavior khi refactor?

2. Định vị trong lộ trình

Slot	Chủ đề	Mối liên hệ
12	REST fundamentals	Biết contract cần bảo vệ
13	Annotations + CRUD	Có code cần test
14	OpenAPI	Có documentation để suy ra test cases
15	Versioning/paging/sorting	Có advanced behavior
16	Testing + MockMvc	Tự động kiểm chứng behavior
17	JPA sâu hơn	Repository test quan trọng hơn
18	REST Controller + Lab 04	Áp dụng strategy quy mô lớn hơn

3. Mục tiêu sau buổi học

1. Phân biệt Unit, web-slice, integration và E2E test theo phạm vi thực sự.
2. Viết controller test với @WebMvcTest + MockMvc.
3. Mock service dependency bằng @MockitoBean (hoặc @MockBean nếu project Boot 3.x cũ).
4. Viết service unit test bằng JUnit 5 + Mockito và hiểu khi nào @SpringBootTest là integration test.
5. Dùng @DataJpaTest đúng cho JPA repository; không dùng nó cho ArrayList repository của core Lab 03.
6. Dùng AssertJ/JUnit assertions, verify interaction, test happy path và error path.
7. Chạy test suite từ IntelliJ và Maven; đọc failure có hệ thống.
8. Hoàn tất Lab 03 và xây testing plan cho Project Assistant 02.

4. Flow tổng quát

```
Requirement / API Contract
↓
Choose SUT + test boundary
↓
Arrange test data / mocks
```

```

↓
Act: call method or perform HTTP request
↓
Assert response / state / interaction
↓
Red → diagnose → fix → Green
↓
Regression suite in Maven / CI

```

5. Câu hỏi dẫn nhập

Scenario: GET /api/employees/E999 hôm nay trả 404. Sau refactor, developer vô tình trả 200 + null. Postman cũ không tự chạy lại. Một MockMvc test có thể phát hiện regression ngay khi `mvn test`.

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. Testing taxonomy – đừng chỉ nhớ tên annotation

Loại/phạm vi	Mục tiêu	Dependency thật?	Tốc độ	Ví dụ Slot 16
Plain unit test	Một class/behavior cô lập	Mock/fake	Rất nhanh	EmployeeService + Mockito
Web slice	Spring MVC layer	Service mock	Nhanh	@WebMvcTest + MockMvc
JPA slice	Entity/repository mapping	Embedded/Test DB	Nhanh-vừa	@DataJpaTest
Full-context integration	Nhiều layer + Spring context	Nhiều bean thật	Vừa-chậm	@SpringBootTest
E2E/live HTTP	Flow qua server thật	Gần production	Chậm	Random port + HTTP client

Important: `@WebMvcTest` thường được gọi “controller test”, nhưng nó không phải plain Java unit test: Spring MVC infrastructure được load theo slice. `@SpringBootTest` load full application context nên là integration-oriented.

2. spring-boot-starter-test cung cấp gì?

- JUnit Jupiter (JUnit 5) để tổ chức/running tests.
- Spring Test + Spring Boot test auto-configuration.
- AssertJ và Hamcrest cho assertions.
- Mockito cho mocks/spies.
- JSON testing utilities và MockMvc support thông qua Spring Test.

pom.xml

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

```

3. JUnit 5 – lifecycle và AAA

```

@BeforeEach
void setUp() {
    // Arrange shared fixtures
}

@Test

```

```

void getEmployee_existingId_returnsEmployee() {
    // Arrange
    // Act
    // Assert
}

@AfterEach
void tearDown() {
    // cleanup only when needed
}

```

Naming habit: Tên test nên mô tả behavior: `method\_condition\_expectedResult`, không phải `test1`, `testGet`.

4. Mockito – mock dependency, không mock SUT

```

@ExtendWith(MockitoExtension.class)
class EmployeeServiceUnitTest {
    @Mock
    IEmployeeRepository repository;

    @InjectMocks
    EmployeeService service;

    @Test
    void getById_existing_returnsEmployee() {
        Employee e = new Employee("E001", "An", "Developer", 15_000_000.0);
        when(repository.getEmployeeById("E001")).thenReturn(e);

        Employee result = service.getEmployeeById("E001");

        assertThat(result).isSameAs(e);
        verify(repository).getEmployeeById("E001");
    }
}

```

4.1 Stub vs verify

Kỹ thuật	Ý nghĩa	Ví dụ
when(...).thenReturn(...)	Điều khiển output của dependency	Repository trả E001
verify(...)	Kiểm tra interaction quan trọng	Service gọi repository đúng 1 lần
ArgumentCaptor	Quan sát argument gửi xuống dependency	Kiểm tra employee trước create
thenThrow(...)	Mô phỏng failure	Repository ném exception

5. MockMvc – test MVC behavior không cần running server

MockMvc đi qua Spring MVC request handling với mock Servlet request/response. Nó kiểm tra routing, binding, JSON conversion, status và exception handling tốt hơn việc gọi trực tiếp controller method.

```

mockMvc.perform(get("/api/employees/E001"))
    .andExpect(status().isOk())
    .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
    .andExpect(jsonPath("$.empId").value("E001"));

```

6. @WebMvcTest + dependency replacement

```

@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
    @Autowired MockMvc mockMvc;

    @MockitoBean

```

```

    IEmployeeService employeeService;
}

```

Compatibility note 2026: Trong Spring Boot 3.4+ `@MockBean` đã deprecated để hướng tới `@MockitoBean`. Nếu project Lab đang ở Boot 3.2/3.3 hoặc slide cũ dùng `@MockBean`, sinh viên có thể gặp annotation cũ; giáo trình ưu tiên `@MockitoBean` cho project Boot 3.4/3.5.

7. Controller test – happy path và error path

```

@Test
void getById_existing_returns200AndJson() throws Exception {
    var e = new Employee("E001", "Nguyen Van A", "Developer", 15_000_000.0);
    when(employeeService.getEmployeeById("E001")).thenReturn(e);

    mockMvc.perform(get("/api/employees/E001"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.empId").value("E001"))
        .andExpect(jsonPath("$.name").value("Nguyen Van A"));
}

@Test
void getById_missing_returns404() throws Exception {
    when(employeeService.getEmployeeById("E999"))
        .thenReturn(new EmployeeNotFoundException("E999"));

    mockMvc.perform(get("/api/employees/E999"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.status").value(404));
}

```

8. POST test – JSON request + response

```

@Autowired ObjectMapper objectMapper;

@Test
void create_valid_returns201() throws Exception {
    var input = new Employee("E004", "Pham Thi D", "HR", 14_000_000.0);
    when(employeeService.createEmployee(any(Employee.class))).thenReturn(input);

    mockMvc.perform(post("/api/employees")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(input)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.empId").value("E004"));
}

```

9. Paging/sorting controller test

```

@Test
void getAll_withPaging_returnsPageContent() throws Exception {
    var content = List.of(
        new Employee("E001", "An", "Developer", 15_000_000.0),
        new Employee("E002", "Binh", "Tester", 12_000_000.0));
    Page<Employee> page = new PageImpl<>(content, PageRequest.of(0, 2), 5);
    when(employeeService.getAllEmployees(any(Pageable.class))).thenReturn(page);

    mockMvc.perform(get("/api/employees")
        .param("page", "0")
        .param("size", "2")
        .param("sortBy", "name")
        .param("direction", "asc"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.content.length()").value(2))
        .andExpect(jsonPath("$.totalElements").value(5));
}

```

10. Service testing – source prompt vs focused practice

Cách	Khi nào dùng	Ưu/nhược
@SpringBootTest + mock repository	Muốn kiểm tra wiring/full context	Chậm hơn; nhiều thứ được load
JUnit + Mockito	Chỉ test business logic service	Nhanh, failure dễ chẩn đoán
@SpringBootTest + repository thật	Integration giữa service và data layer	Thực tế hơn nhưng cần test DB/data isolation

Recommended learning sequence: Trước tiên viết service unit test bằng Mockito để hiểu isolation; sau đó dùng @SpringBootTest khi mục tiêu là integration/wiring. Prompt/slide yêu cầu @SpringBootTest sẽ được minh họa như một lựa chọn integration, không gọi nhầm là pure unit test.

11. Repository testing – core Lab03 và JPA extension phải tách

Repository	Loại test đúng	Lý do
ArrayList EmployeeRepository của Core Lab 03	Plain JUnit test	Không có JPA mapping/DataSource
JpaEmployeeRepository extends JpaRepository	@DataJpaTest	Đây mới là JPA slice; kiểm tra entity/repository/query
Production DB integration	@DataJpaTest + Testcontainers hoặc full integration	Giảm khác biệt H2 vs DB thật

Do not do this: Không gắn `@DataJpaTest` lên repository chỉ dùng `ArrayList` rồi gọi đó là “repository JPA test”. Annotation này dành cho JPA components.

12. @DataJpaTest example cho JPA/H2 extension

```
@DataJpaTest
class JpaEmployeeRepositoryTest {
    @Autowired JpaEmployeeRepository repository;

    @Test
    void saveAndFindById_roundTrip() {
        var saved = repository.save(new EmployeeEntity(
            "E010", "Test User", "Tester", 10_000_000.0));

        var found = repository.findById(saved.getEmpId());

        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("Test User");
    }
}
```

13. E2E awareness – khi nào MockMvc chưa đủ?

- Khi cần kiểm tra server port thật, filter/proxy/external integration hoặc client thật.
- Có thể dùng `@SpringBootTest(webEnvironment = RANDOM\_PORT)` cùng HTTP client; Rest Assured/TestRestTemplate/WebTestClient tùy stack.
- Testcontainers giúp chạy database/service thật trong container cho integration test gần production hơn.
- Slot 16 chỉ cần awareness; core deliverable vẫn là controller/service/repository tests.

14. Những hiểu sai phổ biến

Hiểu sai	Điều chỉnh
Postman pass = không cần automated test	Manual và automated phục vụ mục tiêu khác nhau.
MockMvc chạy server thật	Không; nó dùng mock request/response nhưng đi qua Spring MVC handling.
@SpringBootTest là unit test	Không; nó load full application context.
@DataJpaTest test mọi repository	Chỉ phù hợp JPA components.
Mock mọi thứ	Mock dependency ở boundary; đừng mock behavior bạn đang cần kiểm tra.
100% coverage = chất lượng cao	Coverage không đo chất lượng assertion hay scenario.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

1. Test pyramid cho một REST API nhỏ

E2E / live HTTP	← ít, chậm, critical flows
Integration / slices	← vừa
Unit tests (service/domain)	← nhiều, nhanh

2. Behavior inventory – FUNewsManagementSystem

Behavior	Test layer ưu tiên	Ví dụ assertion
GET news list	Controller slice	200 + JSON array/page
GET missing news	Controller slice	404 + error contract
Create news rule	Service unit	Repository create không được gọi nếu invalid
Paging/sorting	Controller + repository integration	page metadata/order
JPA custom query	@DataJpaTest	Rows đúng theo query
Full auth flow sau này	Integration/E2E	401/403/200 theo role

3. Regression mindset

Mỗi bug thật nên cân nhắc thêm một regression test: tái hiện failure trước, sửa code để test xanh, giữ test trong suite để bug không quay lại.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN THỰC HIỆN VÀ TRIỂN KHAI

A. Demo 1 – Controller Test với @WebMvcTest và MockMvc

A1. Mục tiêu

Test mapping, status, content type, JSON serialization, request binding và error response của EmployeeController mà không chạy server thật.

A2. Skeleton hoàn chỉnh

```
package com.example.employeemanagement.controllers;
import com.example.employeemanagement.pojos.Employee;
```

```

import com.example.employee.management.services.IEmployeeService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.List;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {
    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;
    @MockitoBean IEmployeeService service;

    @Test
    void getAll_returnsPagedData() throws Exception {
        var employees = List.of(
            new Employee("E001", "An", "Developer", 15000000.0),
            new Employee("E002", "Binh", "Tester", 12000000.0));
        when(service.getAllEmployees(any(Pageable.class)))
            .thenReturn(new PageImpl<>(employees, PageRequest.of(0, 10), 2));

        mockMvc.perform(get("/api/employees")
            .param("page", "0").param("size", "10"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.content.length()").value(2));
    }
}

```

If your project uses Boot 3.2/3.3: Thay import `@MockitoBean` bằng
`org.springframework.boot.test.mock.mockito.MockBean`. Không trộn cả hai trong cùng class.

B. Demo 2 – Service Unit Test với Mockito + AssertJ

```

@ExtendWith(MockitoExtension.class)
class EmployeeServiceUnitTest {
    @Mock IEmployeeRepository repository;
    @InjectMocks EmployeeService service;

    @Test
    void create_valid_delegatesToRepository() {
        var e = new Employee("E004", "Dung", "HR", 14000000.0);
        when(repository.create(e)).thenReturn(e);

        var result = service.createEmployee(e);

        assertThat(result).isSameAs(e);
        verify(repository).create(e);
    }

    @Test
    void get_missing_returnsNull_inCurrentLabContract() {
        when(repository.getEmployeeById("E999")).thenReturn(null);
        assertThat(service.getEmployeeById("E999")).isNull();
    }
}

```

Contract note: Nếu Lab 03 đã nâng cấp service để throw `EmployeeNotFoundException`, test phải đổi expected behavior tương ứng. Test phải phản ánh contract hiện tại, không ép code theo snippet.

C. Demo 3 – Service Integration với @SpringBootTest

```
@SpringBootTest
class EmployeeServiceIntegrationTest {
    @Autowired IEmployeeService service;

    @Test
    void applicationContext_wiresServiceAndRepository() {
        assertThat(service.getAllEmployees()).isNotEmpty();
    }
}
```

Đây là integration-style smoke test: mục tiêu là kiểm tra Spring wiring/context. Không thay thế các service unit tests nhanh bằng Mockito.

D. Demo 4 – Core in-memory Repository Test

```
class EmployeeRepositoryTest {
    private EmployeeRepository repository;

    @BeforeEach
    void setUp() {
        repository = new EmployeeRepository();
    }

    @Test
    void create_thenFind_roundTrip() {
        var e = new Employee("E010", "Test User", "Tester", 10000000.0);
        repository.create(e);

        assertThat(repository.getEmployeeById("E010"))
            .usingRecursiveComparison()
            .isEqualTo(e);
    }

    @Test
    void paging_firstPage_returnsExpectedSize() {
        var page = repository.findAll(PageRequest.of(0, 2));
        assertThat(page.getContent()).hasSize(2);
        assertThat(page.getTotalElements()).isGreaterThanOrEqualTo(3);
    }
}
```

E. Demo 5 – JPA Repository Test với @DataJpaTest (extension)

```
@DataJpaTest
class JpaEmployeeRepositoryTest {
    @Autowired JpaEmployeeRepository repository;

    @Test
    void findByDesignation_returnsMatchingRows() {
        repository.save(new EmployeeEntity("E101", "A", "Developer", 1000.0));
        repository.save(new EmployeeEntity("E102", "B", "Tester", 900.0));

        var result = repository.findByDesignation("Developer");

        assertThat(result)
            .extracting(EmployeeEntity::getDesignation)
            .containsOnly("Developer");
    }
}
```

F. Test error cases và validation

Scenario	Expected	Test idea
GET E999	404	service throws/not found contract
POST malformed JSON	400	invalid syntax body
POST missing required name	400 nếu có validation	@Valid + constraint
page=-1	400 hoặc clamp theo contract	assert chosen behavior
sortBy=unknown	400 nếu whitelist	error response
wrong media type	415	contentType text/plain

G. Break-It Lab – chứng minh test có giá trị

1. Đổi controller GET-by-id từ 404 sang `ResponseEntity.ok(null)` . Chạy test: test nào đỏ?
2. Đổi POST status từ 201 thành 200. Chạy suite.
3. Bỏ `@RequestBody` . Quan sát POST test.
4. Đảo sort direction mặc định. Viết/quan sát regression test.
5. Sửa service gọi sai repository method. Dùng `verify()` để phát hiện.
6. Sau mỗi break: revert production code và xác nhận suite xanh.

H. Chạy test từ IntelliJ và Maven

```
# toàn bộ test
mvn test

# một class
mvn -Dtest=EmployeeControllerTest test

# một method (Surefire syntax)
mvn -Dtest=EmployeeControllerTest#getAll_returnsPagedData test
```

H1. Đọc kết quả theo thứ tự

1. Tên test failed.
2. Expected vs actual.
3. Dòng assertion/matcher failed.
4. Root cause đầu tiên trong stack trace thuộc source của mình.
5. Mock setup có match đúng arguments không?
6. Đừng thêm sleep/retry hoặc nới assertion nếu chưa hiểu nguyên nhân.

I. Lab 03 continued – Completion Gate

Gate	Bảng chứng bắt buộc
G1 Build	`mvn test` chạy, không compile error
G2 Controller	GET/POST/PUT/DELETE + error path tests
G3 Paging/Sorting	ít nhất 1 test paging + 1 sort
G4 Service	Business/delegation tests
G5 Repository	In-memory unit tests; JPA slice nếu có JPA extension

G6 Docs	Swagger vẫn truy cập được
G7 Manual	Postman/Swagger smoke tests
G8 Explain	Sinh viên giải thích được boundary của mỗi test

J. Project Assistant 02 – áp dụng cho Group Project

Mỗi nhóm chọn 1 critical resource của project và tạo Testing Strategy Map trước khi code thêm test.

Câu hỏi	Nhóm phải trả lời
Critical behavior?	CRUD nào, rule nào, permission nào quan trọng?
Unit boundary?	Service/domain logic nào test cô lập?
Web slice?	Controller nào cần request/status/JSON test?
Repository?	Query/JPA mapping nào cần @DataJpaTest?
Integration?	Flow nào cần full context/database?
Regression?	Bug nào từng xảy ra cần khóa lại?

V. KINH NGHIỆM THỰC TIỄN, TIPS

Vấn đề	Triệu chứng	Cách xử lý
Test phụ thuộc thứ tự	Chạy riêng pass, chạy suite fail	Mỗi test tự tạo fixture; reset state
Mock setup không match	Mock trả null	Dùng eq()/any() đúng type; inspect args
Over-mocking	Test xanh dù integration hỏng	Chỉ mock external/lower-layer dependency
Brittle JSON assertion	Thêm field là test hỏng	Assert contract quan trọng thay vì toàn bộ JSON string
Full context mọi test	Suite chậm	Ưu tiên unit/slice tests
@DataJpaTest với ArrayList repo	Context/query vô nghĩa	Dùng plain JUnit cho in-memory repo
Test implementation detail	Refactor làm hàng loạt test hỏng	Assert observable behavior
Không test error path	Production fail ở edge cases	Mỗi endpoint có negative scenarios

Testing heuristic: FIRST

- Fast – test đủ nhanh để chạy thường xuyên.
- Independent – không phụ thuộc test trước.
- Repeatable – kết quả ổn định ở máy khác.
- Self-validating – pass/fail rõ, không cần nhìn thủ công.
- Timely – viết gần thời điểm behavior được phát triển/sửa bug.

Không chạy theo coverage một cách cơ học

Coverage là tín hiệu phụ. Một test suite tốt ưu tiên critical behavior, boundary/error cases, contract và regression risk; 90% coverage với assertions yếu có thể kém hơn 70% coverage đúng trọng tâm.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

A. Bài tập chủ đề

Bài	Yêu cầu	Evidence
1. Controller GET	Test list + by-id + 404	MockMvc output
2. POST	201 + JSON body	Test pass
3. DELETE	Success + missing	Status assertions
4. Paging	page/size metadata	jsonPath assertions
5. Sorting	asc/desc behavior	Test + explanation
6. Service	Mockito + verify	Unit tests
7. In-memory repo	CRUD/paging plain JUnit	No Spring context
8. JPA extension	@DataJpaTest custom query	H2 slice
9. Break-It	1 red→green regression	Screenshot/log
10. Strategy	Testing map for FUNews/group project	Worksheet

B. Bài tập tổng hợp – Hoàn thành Lab 03 với Testing

- Giữ Employee Management API hoạt động.
- Controller tests: GET list, GET by ID, POST, PUT/DELETE nếu project có, 404/error case.
- Paging/sorting tests.
- Service unit tests bằng Mockito; optional full-context smoke test.
- Repository tests đúng loại: plain JUnit cho in-memory; @DataJpaTest nếu làm JPA/H2 extension.
- Chạy `mvn test` và đảm bảo toàn suite pass.
- Gây ít nhất một lỗi có chủ đích, chụp red test, sửa và chụp green test.
- Cập nhật README mục Testing: cách chạy, phạm vi, known limitations.

C. Rubric mini cho bài tổng hợp

Tiêu chí	Điểm
Controller/MockMvc tests đúng contract	25
Service tests và mocking đúng	20
Repository tests đúng boundary	15
Paging/sorting/error cases	15
Readability + naming + AAA	10
Red→Green evidence + debug explanation	10
README/testing commands	5

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
Unit Test	Kiểm tra một đơn vị nhỏ trong isolation.	EmployeeService + mocked repository
Integration Test	Kiểm tra nhiều thành phần tương tác.	@SpringBootTest
E2E Test	Kiểm tra flow qua hệ thống chạy thật.	HTTP tới random port
Test Slice	Chỉ load một phần Spring context.	@WebMvcTest/@DataJpaTest
SUT	System Under Test.	EmployeeService
Test Fixture	Dữ liệu/setup phục vụ test.	Employee E001
JUnit 5	Framework test Java.	@Test, @BeforeEach
Mockito	Mocking framework.	@Mock, when, verify
AssertJ	Fluent assertions.	assertThat(...)
MockMvc	Spring MVC test framework không cần server thật.	perform(get(...))
@WebMvcTest	MVC slice test.	EmployeeControllerTest
@SpringBootTest	Full application context test.	Integration smoke test
@DataJpaTest	JPA slice test.	JpaEmployeeRepositoryTest
@MockitoBean	Spring test bean override bằng Mockito mock.	Mock service trong @WebMvcTest
@MockBean	Boot annotation cũ, deprecated 3.4+.	Legacy compatibility
@Mock	Mockito mock ngoài Spring context.	Repository in service unit test
@InjectMocks	Inject Mockito mocks vào SUT.	EmployeeService
Stub	Cấu hình output dependency.	when(...).thenReturn(...)
Verify	Kiểm tra interaction.	verify(repo).create(e)
MockMvc request builder	Tạo mock HTTP request.	get/post/put/delete
andExpect	Assert MockMvc response.	status().isOk()
jsonPath	Assert JSON structure/value.	\$.content.length()
Happy Path	Scenario thành công bình thường.	GET E001 → 200
Error Case	Scenario invalid/failure.	GET E999 → 404
Regression Test	Test khóa một bug đã sửa.	POST status 201
Test Coverage	Tỷ lệ code được execute bởi test.	Jacoco metric
Rest Assured	DSL cho REST API testing.	given/when/then
Testcontainers	Containerized dependencies cho test.	PostgreSQL test container
Red-Green-Refactor	Fail test → pass → cải tiến.	Break-It lab
Flaky Test	Test lúc pass lúc fail.	Shared state/time dependency

VIII. RESEARCH BASE VÀ AI BASE LEARNING

A. 10 prompt nghiên cứu chuyên sâu

1. Phân tích khác nhau giữa plain unit test, @WebMvcTest, @DataJpaTest và @SpringBootTest theo context được load.
2. Giải thích MockMvc đi qua DispatcherServlet như thế nào mà không cần server thật.
3. Phân tích @MockitoBean và @MockBean: lifecycle, deprecation và migration.
4. Giải thích khi nào verify interaction hữu ích và khi nào khiến test brittle.
5. Phân tích cách ObjectMapper/Jackson ảnh hưởng controller tests.
6. Giải thích rollback semantics của @DataJpaTest.
7. Phân tích test double: dummy, stub, mock, fake, spy.
8. Giải thích mutation testing và vì sao coverage cao chưa đủ.
9. Phân tích contract testing giữa frontend/backend hoặc microservices.
10. Giải thích Testcontainers giúp giảm khác biệt H2 và production DB như thế nào.

B. 10 prompt thực tiễn production

1. Thiết kế testing pyramid cho một Spring Boot REST service có PostgreSQL và Redis.
2. Trong CI/CD, test nào chạy ở pull request, test nào chạy nightly?
3. Làm sao test pagination/sorting mà không làm test phụ thuộc thứ tự dữ liệu ngẫu nhiên?
4. Làm sao test error response contract nhất quán từ @ControllerAdvice?
5. Khi nào nên dùng Testcontainers thay cho H2?
6. Làm sao giảm thời gian của một suite có quá nhiều @SpringBootTest?
7. Làm sao test API có authentication/authorization bằng MockMvc?
8. Làm sao quản lý test data để tests independent và repeatable?
9. Làm sao viết regression test từ một production incident?
10. Làm sao review chất lượng test code trong code review?

C. 10 prompt xu hướng/cập nhật

1. So sánh MockMvc, MockMvcTester và WebTestClient cho Spring MVC hiện đại.
2. So sánh Rest Assured với Spring HTTP interface testing.
3. JUnit 5/6 có những thay đổi nào ảnh hưởng Spring testing?
4. Testcontainers hiện được tích hợp với Spring Boot như thế nào?
5. Consumer-driven contract testing với Pact/Spring Cloud Contract là gì?
6. Mutation testing với PIT phù hợp Spring Boot ra sao?
7. Property-based testing cho REST API là gì?
8. AI-generated tests có rủi ro assertion hallucination như thế nào?
9. Flaky test detection trong CI hiện đại được triển khai thế nào?
10. Observability-driven testing và production verification khác test truyền thống ra sao?

D. AI Verification Worksheet

Prompt/claim AI	Nguồn kiểm chứng	Thử nghiệm	Kết luận của SV
AI nói @SpringBootTest là unit test	Spring docs + context load	So tốc độ/context	
AI dùng @DataJpaTest cho ArrayList repo	Spring Boot docs	Run/inspect	
AI đề xuất @MockBean	Boot version/deprecation	Compile/import	
AI tạo controller test	Run red/green	Break production code	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Self-check kiến thức

- Unit test khác integration test ở boundary nào?
- @WebMvcTest load những gì và không load những gì?
- MockMvc có chạy server thật không?
- Khi nào dùng @MockitoBean và khi nào dùng @Mock?
- Vì sao @SpringBootTest chậm hơn service unit test?
- @DataJpaTest có phù hợp ArrayList repository không?
- Stub và verify khác nhau thế nào?
- Một controller test tốt nên assert những gì ngoài status code?
- Test paging cần kiểm tra những metadata/behavior nào?
- Flaky test là gì và vì sao nguy hiểm?

2. Checklist kỹ năng

Nội dung	Đạt	Chưa đạt	Làm lại
Chạy được mvn test	☐	☐	☐
Viết được @WebMvcTest	☐	☐	☐
Dùng được MockMvc	☐	☐	☐
Dùng được @MockitoBean/@MockBean đúng version	☐	☐	☐
Viết được Mockito service unit test	☐	☐	☐
Dùng được AssertJ	☐	☐	☐
Viết được in-memory repository test	☐	☐	☐
Dùng đúng @DataJpaTest cho JPA extension	☐	☐	☐
Test được 404/error cases	☐	☐	☐
Test được paging	☐	☐	☐
Test được sorting	☐	☐	☐
Đọc được stack trace	☐	☐	☐
Thực hiện được Red→Green regression test	☐	☐	☐

Giải thích được test boundary	☐	☐	☐
-------------------------------	--------------------------	--------------------------	--------------------------

3. Mức tự đánh giá

Mức	Biểu hiện
Mức 1 – Chưa đạt	Chỉ chạy Postman; chưa viết/giải thích được automated tests.
Mức 2 – Đạt cơ bản	Viết được MockMvc + service tests, chạy suite pass.
Mức 3 – Tốt	Chọn đúng test boundary, có negative/regression tests, phân biệt slice/full-context/JPA và áp dụng cho project khác.

4. Exit Ticket

1. Một test nào hôm nay giúp bạn hiểu HTTP/MVC rõ hơn?
2. Một trường hợp bạn từng nghĩ cần @SpringBootTest nhưng nay có thể dùng unit/slice test?
3. Một rủi ro testing bạn sẽ kiểm tra trong Group Project?

PHỤ LỤC A – 135 PHÚT GỢI Ý CHO GIẢNG VIÊN

Thời gian	Hoạt động
0–15	Retrieval Slot 15 + testing scenario
15–35	Testing taxonomy + JUnit/Mockito/AssertJ
35–60	Live @WebMvcTest + MockMvc
60–80	Service unit tests + @SpringBootTest comparison
80–98	Repository tests: in-memory vs @DataJpaTest
98–115	Break-It Lab: Red → Green
115–128	Lab 03 completion + Project Assistant 02
128–135	Human Gate + Exit Ticket

PHỤ LỤC B – TESTING STRATEGY WORKSHEET

Behavior/Requirement	SUT	Test type	Dependency real/mock	Input	Expected	Evidence

PHỤ LỤC C – HUMAN VERIFICATION GATE

1. Giải thích vì sao MockMvc test controller tốt hơn gọi trực tiếp method khi muốn kiểm tra mapping/binding.
2. Mở EmployeeControllerTest và chỉ ra đâu là Arrange, Act, Assert.
3. Nếu bỏ @RequestBody khỏi POST controller, test nào phải fail và vì sao?
4. @MockitoBean khác @Mock ở chỗ nào?
5. Tại sao @SpringBootTest không nên là mặc định cho mọi test?
6. Vì sao @DataJpaTest không phù hợp ArrayList repository của core Lab 03?
7. Viết một negative test cho E999.
8. Test paging page=0,size=2 nên assert gì?
9. Test sorting cần làm sao để chắc chắn thứ tự đúng?
10. Khi mock trả null ngoài ý muốn, bạn kiểm tra gì trước?
11. Khi nào verify() làm test tốt hơn? Khi nào khiến test brittle?
12. Tại sao test phải independent?
13. Phân biệt bug trong production code và bug trong test.
14. Tạo một regression test từ bug POST trả 200 thay vì 201.
15. Giải thích một test nào AI đề xuất nhưng bạn đã sửa sau khi kiểm chứng.

PHỤ LỤC D – EVIDENCE PACKAGE & GIT CHECKPOINTS

ID	Evidence
E1	Baseline Lab 03 API/Swagger chạy
E2	Controller test source + green run
E3	Service unit test source + green run
E4	Repository test source
E5	Paging test
E6	Sorting test
E7	Error case test
E8	Red test screenshot/log
E9	Green after fix
E10	Maven test summary
E11	Testing Strategy Worksheet
E12	AI verification log
E13	README Testing section
E14	Human Gate notes

Suggested checkpoints

```
V0 baseline-lab03
V1 controller-tests
V2 service-unit-tests
V3 repository-tests
```

V4 paging-sorting-error-tests
 V5 red-green-regression
 V6 lab03-complete-testing

PHỤ LỤC E – QUICK REFERENCE

Mục tiêu	Tool/annotation	Nhớ
MVC controller slice	@WebMvcTest + MockMvc	Không cần live server
Mock Spring bean	@MockitoBean	Boot 3.4+ preferred
Legacy Spring bean mock	@MockBean	Deprecated từ Boot 3.4
Plain unit mock	@Mock + @InjectMocks	Không cần Spring context
Full context	@SpringBootTest	Integration-oriented
JPA slice	@DataJpaTest	JPA only; transactional/rollback
Readable assertions	AssertJ	assertThat(...)
Mock behavior	when/thenReturn	Stub dependency
Interaction	verify	Dừng khi interaction là requirement

TÀI LIỆU THAM KHẢO VÀ GHI CHÚ TƯƠNG THÍCH

- Slide Chapter 12, trang 20–42 và prompt Slot 16 của môn học.
- Spring Framework Reference – MockMvc: kiểm tra Spring MVC bằng mock request/response mà không cần server thật.
- Spring Boot 3.5 Reference/API – @WebMvcTest tập trung MVC components và thường kết hợp @MockitoBean.
- Spring Boot 3.4+ – @MockBean deprecated, migration sang Spring Framework @MockitoBean.
- Spring Boot – @DataJpaTest là JPA test slice; chỉ dùng cho JPA extension của Lab 03.
- Lab 03 – Employee Management API là ngữ liệu thực hành xuyên suốt.

Scope lock: Slot 16 không dạy lại sâu versioning/paging/sorting; các nội dung đó chỉ xuất hiện như behavior cần test. JPA sâu hơn tiếp tục ở Slot 17.